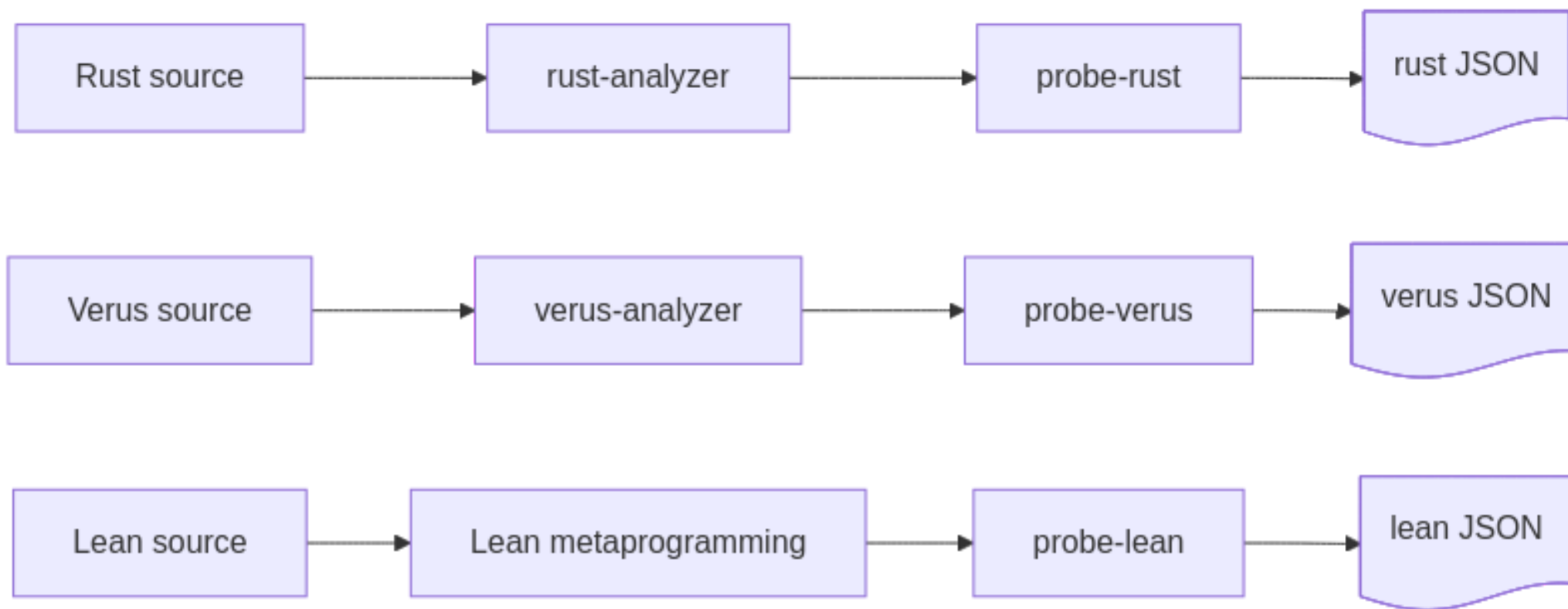


Probes: factual data about (verified) code

A small tool ecosystem that reads what code indexers understand and writes it down as JSON.

What the probes are

Code indexers already parse and understand a codebase. The probes read that understanding and emit **structured, factual data** — no interpretation, no styling.



probe-aeneas has no indexer of its own: it runs probe-rust and probe-lean and *joins* their

The JSON: one entry per code atom

Every probe emits the same shape — one entry per atom (a Rust function, a Verus construct, a Lean construct) with its dependencies and status.

Project	Typical information per atom
Rust	function calls (the call graph)
Verus, Aeneas	calls + verification status; thm deps + proof status
Lean	theorem dependencies + proof status

A Verus atom

A Rust function, its calls, and whether it verifies against its spec.

```
"probe:curve25519-dalek/.../[ProjectivePoint]double()": {  
  "kind": "exec",  
  "language": "rust",  
  "primary-spec": "requires is_valid_projective_point(*self), ensures ...",  
  "verification-status": "transitively-verified",  
  "dependencies": [  
    "probe:.../[FieldElement51]square()",  
    "probe:.../[FieldElement51]square2()" ]  
}
```

probe-aeneas: Rust ↔ Lean, linked

An Aeneas project has two sides: the **Rust crate** and the **Aeneas-generated Lean** that models it plus the specs proved about it.

- **probe-rust** indexes the crate and tags each function with a **Charon-derived qualified name**.
- **probe-lean** indexes the Lean side, where each generated def remembers the Rust function it came from.

Charon names are the **shared vocabulary**: matching them links a Rust function to the Lean def that implements it and the theorem that specifies it.

Where spec and proof live differ by tool

Same property (`FieldElement51::is_negative`), verified both ways.

Verus	Lean via Aeneas
Code, spec and proof are one artifact	Code is a plain <code>def</code> ; spec + proof are a separate theorem
spec = <code>requires</code> / <code>ensures</code> contract	spec = the theorem's statement
proof = <code>proof { }</code> inside the fn	proof = the proof term

This split is why "what does a Lean `def` mean?" is harder than in Verus — a `def` can be an impl, a spec, or neither.

A Lean `def` is multi-faceted

Verus has dedicated syntax (`spec fn` vs `fn`). Lean has only `def` — one keyword for an implementation, a spec, or something else. **The syntax is identical**, so in general we cannot tell a spec-def from an impl-def.

Consequence: **no single colour per `def`** — it depends on the project.

- **Aeneas projects:** for Aeneas-generated translations (they model implementations) they get the colour of the specs, **blue** for defs that play the role of a Verus `spec fn`.
- **Mathlib-style projects:** there is a proposal to have a **green** dot to denote the def "compiles", not "meets a spec".
- **Verification projects done in Lean (like lean-zip):** without annotations we cannot distinguish defs representing impls from def representing specs

Three kinds of projects, three questions

Functional
verification

$f \models \text{spec}$

"does f meet
its spec?"

Mathlib-style
formalization

$\vdash \text{thm}$

"is thm
proved?"

Security
protocol (Lean)

$\text{AEAD} \models \text{secure}$

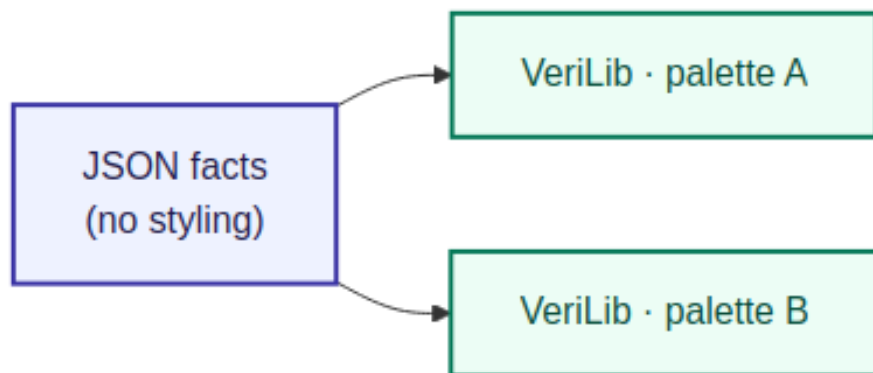
"is the AEAD
construction
secure?"

Different questions in nature — forcing all three into one framework loses meaning.

Division of labour

The **probes** have one job: report factual data about the code, as JSON. They take **no position** on how it should look.

VeriLib takes that data and presents it — by colouring atoms and computing statistics.



What colours and which stats are **subjective**: helpful to one user, "nay" to another. So the colour/status choices below are a VeriLib concern to reach consensus on — not facts the probes emit.

Currently

In VeriLib we see:

Statuses:

STATUS

Disabled (0)

Validated (0)

Verified (0)

Not Validated (0)

Colors:

- Not Started
- Specs (no proofs)
- Verified (proofs)
- Total tracked functions
- Transitivity Verified
- Translated

There's a gap between them (we should have a 1-to-1 correspondence between statuses and colours)

Statuses

The probes don't (cannot) emit info about:

- a function being tracked
- a spec being validated

In the current verif projects:

- specs are validated through PRs, if a spec exists in the codebase, then it's validated
- there's nothing in the code saying that a function is tracked

Colour ↔ status proposal

A one-to-one mapping between verification status and colour:

- **disabled** — no spec / outside verification scope
- **translated** — Aeneas def with no spec yet
- **sorry / assumes**
- **error**
- **verified** — a function meets its spec, or a theorem is proved
- **trusted** — axioms / assumed

Shapes

Colours for statuses, shapes for roles:

- can be def/thm like in verso-blueprint; not sure if it makes sense for verus...
- can be impl/spec/proof; could make sense for Verus; not much for Lean spec-theorems; maybe just impl/spec? just have a shape for specs?

What we have in the jsons:

- kinds: def/abbrev/thm/... for lean; exec/spec/proof for verus

Arrows

The following fields in the jsons from the probes can allow us to have different arrows:

- `translation-name`
- `primary-spec`
- `dependencies`

Open question 1 — green

Should green mean *only* "satisfies its spec / is proved"?

- If yes, **arbitrary Lean** `def` s get no verification colour ("this def compiles" is diff from "this function meets its spec").
- Aeneas-generated defs still get a colour: **translated** until specified, then the colour of their spec.
- for vcvio based projects, we can leverage info like [Jin implemented](#)
- for verso-blueprint projects, we have a probe wrapping around verso-blueprint to get all that the user annotated

Open question 2 — blue for specs

Do we want blue for specs (the spec itself, not a specified function)?

- If a function has a spec, it also has a proof → the outcome is already green/orange/red. Blue is "eaten".
- Only genuine case: **spec defs** used in pre/post-conditions (Verus `spec fn`, similar Aeneas `defs`).

Or we could see a spec as a **role** → maybe distinguish specs by **shape**, not colour.

Open question 3 — tracking

Is every Rust function "tracked" by default?

- **Yes** → a finished project needs an `outside_verif_scope` annotation, else it shows as incomplete (has an upper bound).
- **No** (current) → a function with no spec is disabled; no upper bound, but the progress chart still grows as specs are added.

Open question 4 — pure Rust projects

Should VeriLib display pure Rust (unverified) projects at all?

- If yes, what colour are those atoms?
- white (as neutral) conveys "not for verification" — but today white is *also* used for tracked functions, which is inconsistent.

Open question 5 — "done"

When is a project done?

- Verification: when Rust functions are **green**?
- Formalization: when theorems are **green**?
- Without a notion of tracking, **any intermediate state looks "done"**.

Open question 6 — trusted / axioms

Distinguish axioms by colour (purple) or by shape?

- Colour lets us **quantify** how much is trusted (how much purple).
- Shape does not quantify as easily.